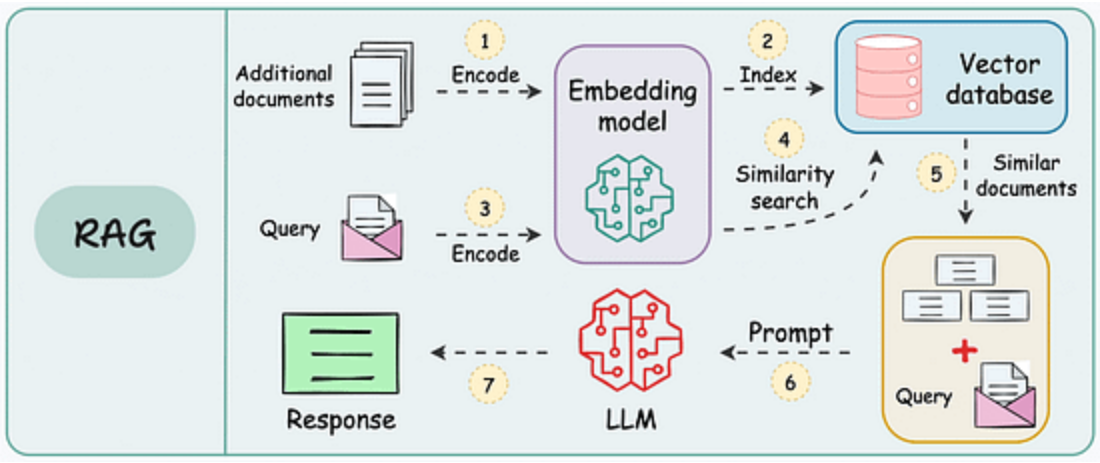
5 Chunking Strategies For RAG

...explained in a single frame.



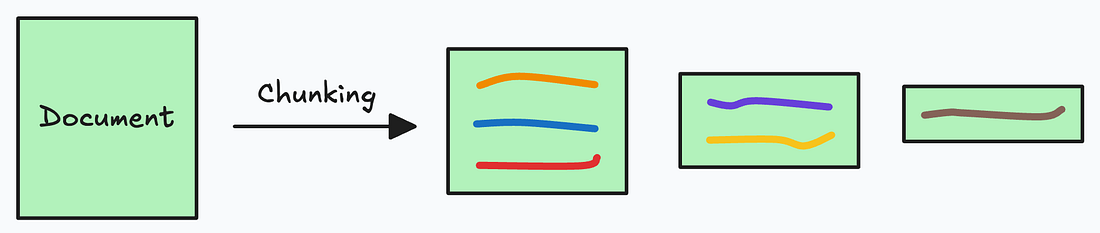
AVI CHAWLA
OCT 18, 2024

Here's the typical workflow of a RAG application:



RAG: Store additional information as vectors, match the incoming query to those vectors, and feed the most similar info to the LLM along with the query.

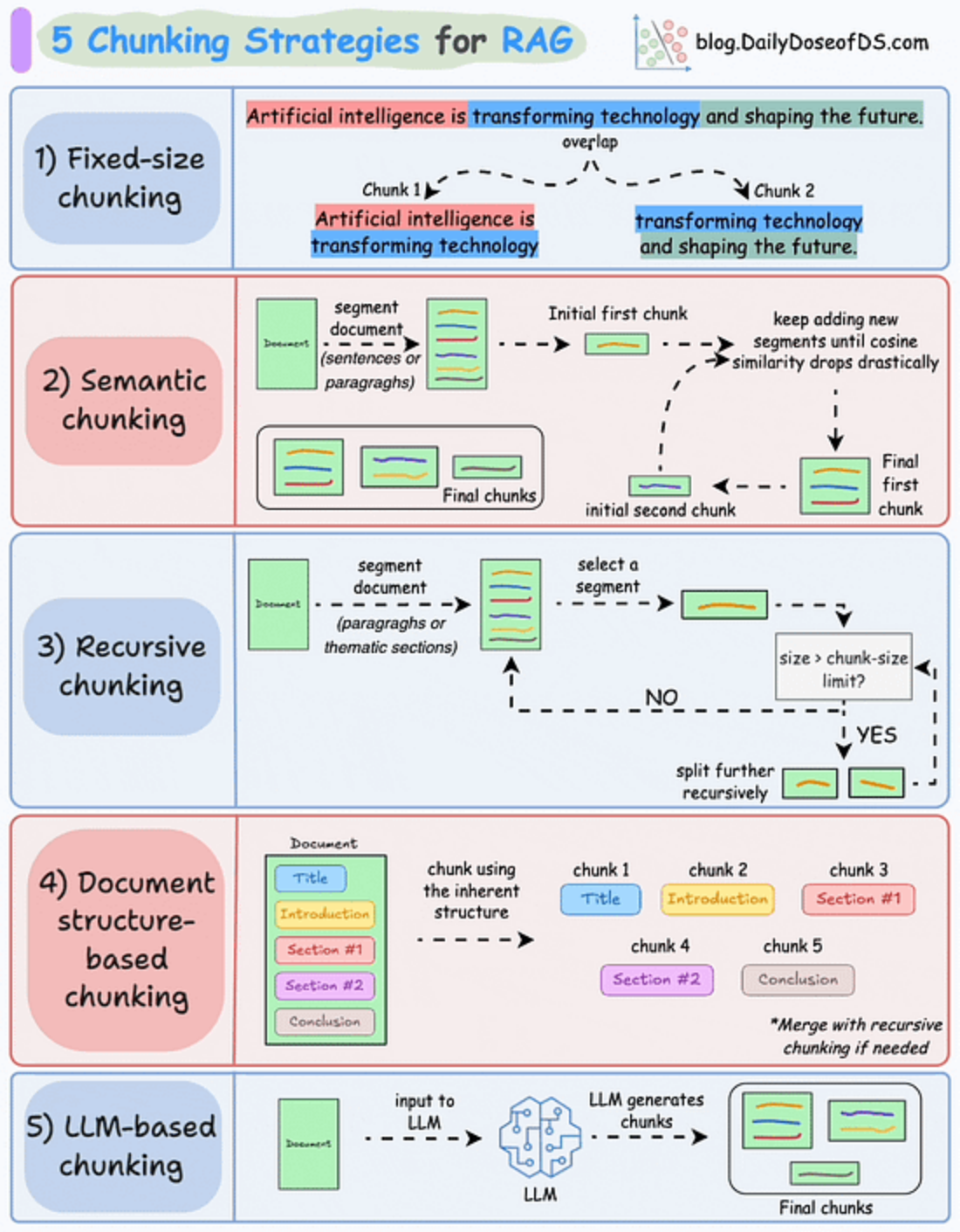
Since the additional document(s) can be pretty large, step 1 also involves chunking, wherein a large document is divided into smaller/manageable pieces.



This step is crucial since it ensures the text fits the input size of the embedding model.

Moreover, it enhances the efficiency and accuracy of the retrieval step, which directly impacts the quality of generated responses (we discussed this yesterday).

Here are five chunking strategies for RAG:



Let's understand them today!

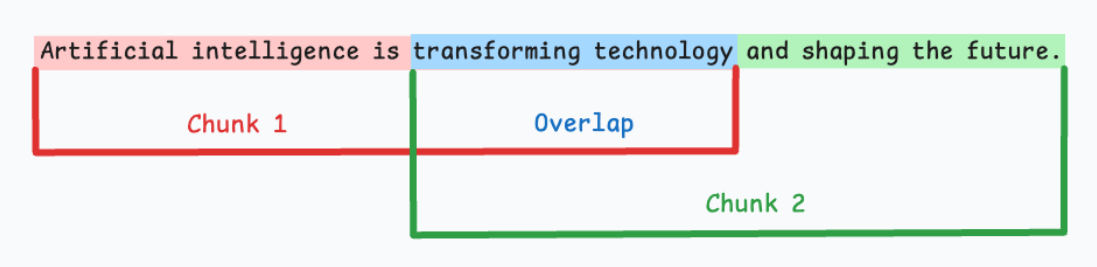
> *Note: Yesterday, we discussed techniques to build robust NLP systems that rely on pairwise content similarity (RAG is one of them). Read here in case*

> *you missed it:* ***Bi-encoders and Cross-encoders for Sentence Pair Similarity Scoring – Part 1****.*

---

1) Fixed-size chunking

The most intuitive and straightforward way to generate chunks is by splitting the text into uniform segments based on a pre-defined number of characters, words, or tokens.



Since a direct split can disrupt the semantic flow, it is recommended to maintain some overlap between two consecutive chunks (the blue part above).

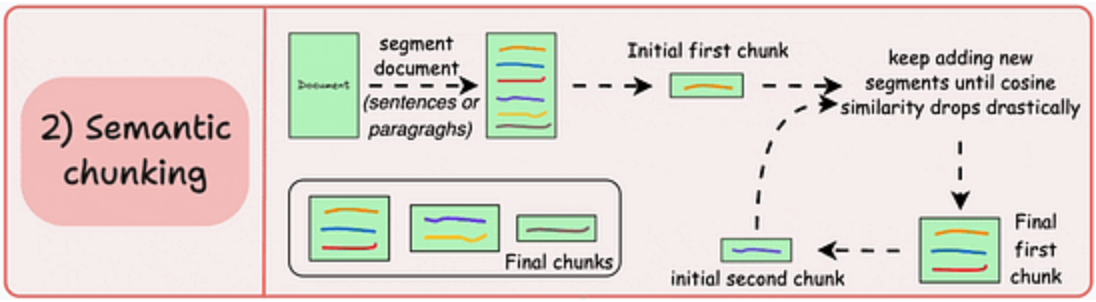
This is simple to implement. Also, since all chunks are of equal size, it simplifies batch processing.

But there is a big problem. This usually breaks sentences (or ideas) in between. Thus, important information will likely get distributed between chunks.

---

2) Semantic chunking

The idea is simple.



- Segment the document based on meaningful units like sentences, paragraphs, or thematic sections.
- Next, create embeddings for each segment.
- Let's say I start with the first segment and its embedding.
    - If the first segment's embedding has a high cosine similarity with that of the second segment, both segments form a chunk.
    - This continues until cosine similarity drops significantly.
    - The moment it does, we start a new chunk and repeat.

Here's what the output could look like:

```
Artificial intelligence is transforming industries by automating
processes, enhancing decision-making, and providing insights through
data analysis. Machine learning, a subset of AI, enables systems to learn
and improve from experience without explicit programming. Deep learning,
a branch of machine learning, uses neural networks with multiple layers
to model complex patterns in data.
```

Unlike fixed-size chunks, this maintains the natural flow of language and preserves complete ideas.

Since each chunk is richer, it improves the retrieval accuracy, which, in turn, produces more coherent and relevant responses by the LLM.

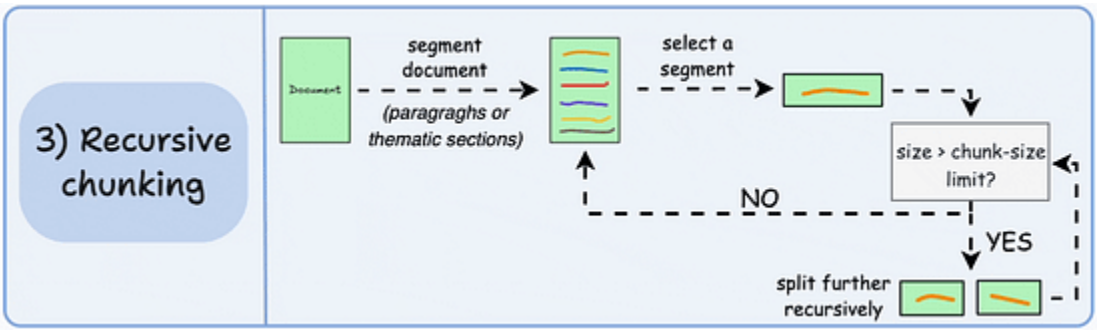
A minor problem is that it depends on a threshold to determine if cosine similarity has dropped significantly, which can vary from document to

document.

---

3) Recursive chunking

This is also simple.



First, chunk based on inherent separators like paragraphs, or sections.

Next, split each chunk into smaller chunks if the size exceeds a pre-defined chunk size limit. If, however, the chunk fits the chunk-size limit, no further splitting is done.

Here's what the output could look like:

```
Paragraph 1

Artificial intelligence is transforming industries by automating
processes, enhancing decision-making, and providing insights through
data analysis. Machine learning, a subset of AI, enables systems to learn
and improve from experience without explicit programming. Deep learning,
a branch of machine learning, uses neural networks with multiple layers
to model complex patterns in data.

Paragraph 2

AI is also improving natural language processing, enabling applications
like chatbots and virtual assistants.
```

As shown above:

- First, we define two chunks (the two paragraphs in purple).
- Next, paragraph 1 is further split into smaller chunks.

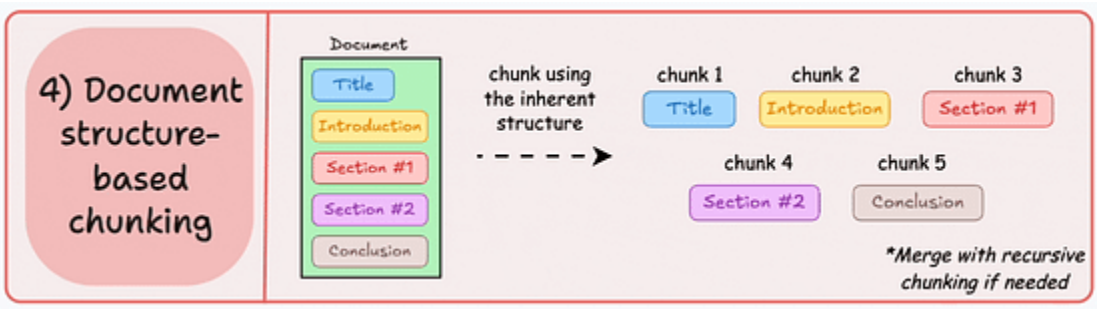
Unlike fixed-size chunks, this approach also maintains the natural flow of language and preserves complete ideas.

However, there is some extra overhead in terms of implementation and computational complexity.

---

4) Document structure-based chunking

This is another intuitive approach.



It utilizes the inherent structure of documents, like headings, sections, or paragraphs, to define chunk boundaries.

This way, it maintains structural integrity by aligning with the document's logical sections.

Here's what the output could look like:

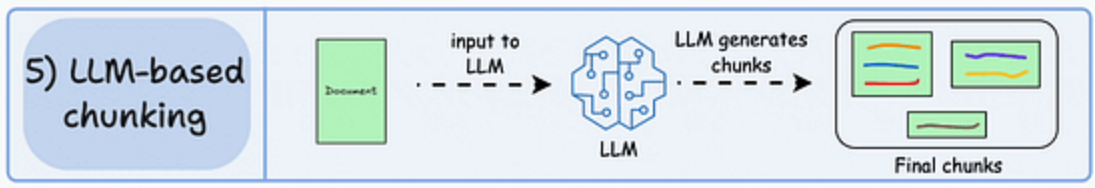
Title: The Role of Artificial Intelligence in Modern Education	Chunk 1
Introduction Artificial intelligence (AI) is reshaping education by providing personalized learning experiences and automating administrative tasks.	Chunk 2
Section 1: Personalized Learning AI enables the customization of educational content to meet individual student needs, enhancing engagement and comprehension.	Chunk 3
Section 2: Administrative Automation From grading to scheduling, AI tools are streamlining administrative processes, allowing educators to focus more on teaching.	Chunk 4
Conclusion The integration of AI in education holds the promise of more efficient learning environments and improved student outcomes.	Chunk 5

That said, this approach assumes that the document has a clear structure, which may not be true.

Also, chunks may vary in length, possibly exceeding model token limits. You can try merging it with recursive splitting.

---

5) LLM-based chunking



Since every approach has upsides and downsides, why not use the LLM to create chunks?

The LLM can be prompted to generate semantically isolated and meaningful chunks.

Quite evidently, this method will ensure high semantic accuracy since the LLM can understand context and meaning beyond simple heuristics (used in the above four approaches).

The only problem is that it is the most computationally demanding chunking technique of all five techniques discussed here.

Also, since LLMs typically have a limited context window, that is something to be taken care of.

Each technique has its own advantages and trade-offs.

I have observed that semantic chunking works pretty well in many cases, but again, you need to test.

The choice will heavily depend on the nature of your content, the capabilities of the embedding model, computational resources, etc.

We shall be doing a hands-on demo of these strategies pretty soon.

In the meantime, in case you missed it, yesterday, we discussed techniques to build robust NLP systems that rely on pairwise content similarity (RAG is one of them).

Read here: **Bi-encoders and Cross-encoders for Sentence Pair Similarity Scoring – Part 1**.

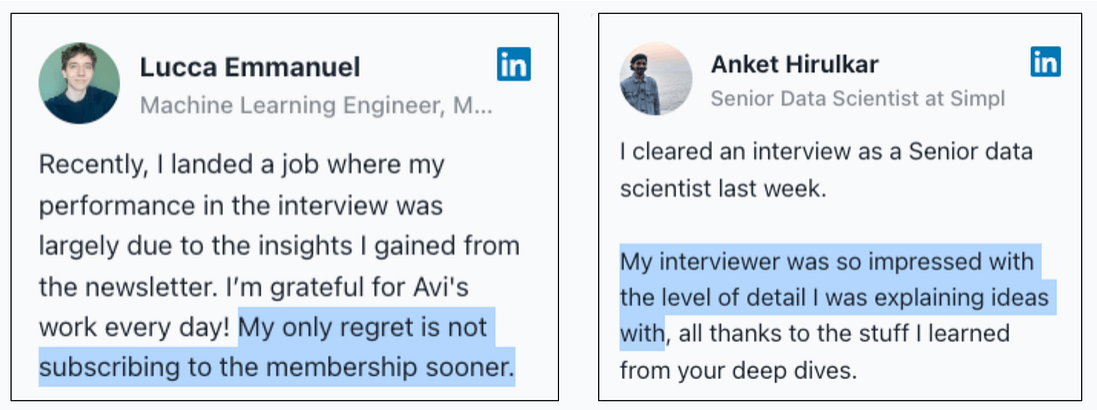
👉 Over to you: What other chunking strategies do you know?

Thanks for reading Daily Dose of Data Science! Subscribe below and receive a free

data science PDF (530+ pages) with 150+ core data science and machine learning lessons.

Subscribed

P.S. For those wanting to develop "Industry ML" expertise:



At the end of the day, all businesses care about *impact*. That's it!

- Can you reduce costs?
- Drive revenue?
- Can you scale ML models?
- Predict trends before they happen?

We have discussed several other topics (with implementations) in the past that align with such topics.

Develop "Industry ML" Skills

Here are some of them:

- Learn sophisticated graph architectures and how to train them on graph data: A Crash Course on Graph Neural Networks – Part 1
- Learn techniques to run large models on small devices: Quantization: Optimize ML Models to Run Them on Tiny Hardware
- Learn how to generate prediction intervals or sets with strong statistical guarantees for increasing trust: Conformal Predictions: Build Confidence in Your ML Model's Predictions.
- Learn how to identify causal relationships and answer business questions: A Crash Course on Causality – Part 1
- Learn how to scale ML model training: A Practical Guide to Scaling ML Model Training.
- Learn techniques to reliably roll out new models in production: 5 Must-Know Ways to Test ML Models in Production (Implementation Included)
- Learn how to build privacy-first ML systems: Federated Learning: A Critical Step Towards Privacy-Preserving Machine Learning.
- Learn how to compress ML models and reduce costs: Model Compression: A Critical Step Towards Efficient Machine Learning.

All these resources will help you cultivate key skills that businesses and companies care about the most.

SPONSOR US

Get your product in front of 100,000 data scientists and other tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters — thousands of leaders, senior data scientists,

machine learning engineers, data analysts, etc., who have influence over significant tech decisions and big purchases.

To ensure your product reaches this influential audience, reserve your space **here** or reply to this email to ensure your product reaches this influential audience.